

Universidade Federal de São Carlos - UFSCar
Centro de Ciências Exatas e de Tecnologia
Departamento de Computação

**Relatório de Prática: Comunicação Serial entre
Arduino Uno e Microcontrolador 8051 para Controle
de Display LCD**

Gabriel Mello Silveira Targas - 812990
Guilherme Barbosa - 811692

Professor: Dr. Emerson Carlos Pedrino

São Carlos, SP
2026

Sumário

1	Introdução	2
2	Arquitetura e Conexões Físicas	2
3	Implementação e Análise dos Códigos	3
3.1	Código do Arduino Uno (Transmissor)	4
3.2	Código do 8051 (Receptor e Controlador do LCD)	4
4	Resultados e Conclusão	7
5	Referências	7

1 Introdução

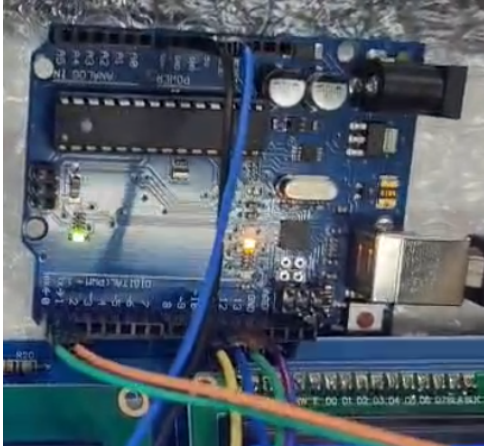
O presente relatório documenta a implementação de uma arquitetura de comunicação serial assíncrona (UART) envolvendo um Arduino Uno e um microcontrolador da família 8051 (89S52). O objetivo da prática é transmitir uma sequência específica de bytes a partir do Arduino para o 8051, que atua como receptor e gerencia a exibição destes dados em um display LCD (Liquid Crystal Display) operando em modo de 4 bits.

Um foco específico desta implementação é a correta renderização de caracteres não contidos na tabela ASCII padrão, demonstrando o acesso direto à memória ROM do controlador do display (como o HD44780) via envio de dados em formato hexadecimal.

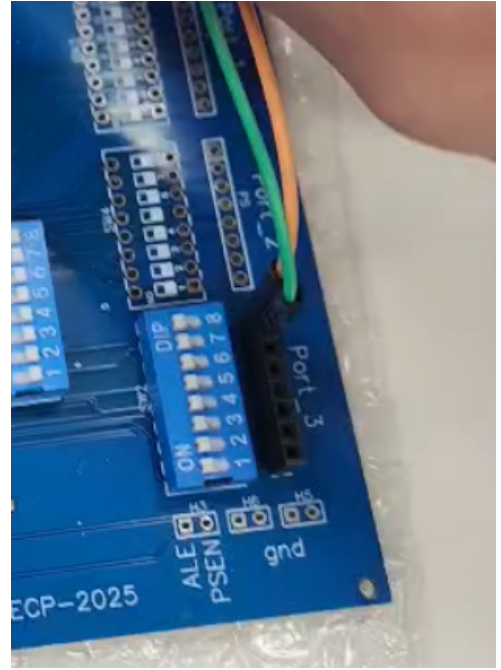
2 Arquitetura e Conexões Físicas

Como as características fundamentais da placa base e do módulo LCD já foram exploradas em práticas anteriores, a montagem prioriza as vias de dados e comunicação.

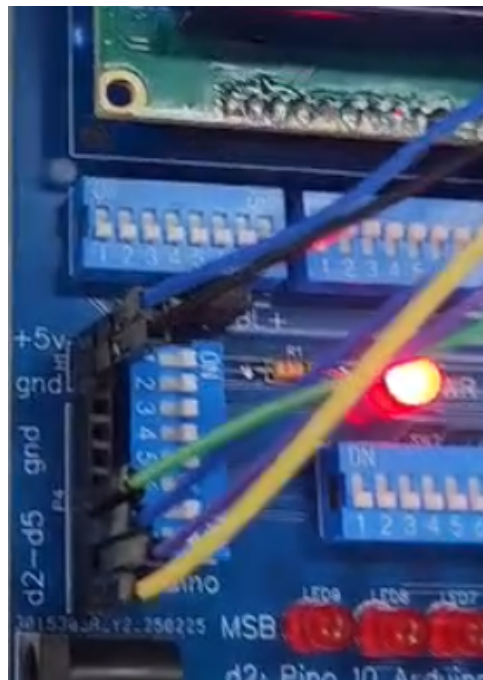
A arquitetura foi estabelecida configurando o Arduino Uno como transmissor (TX) e o 8051 como receptor (RX). A interface com o LCD foi realizada dividindo o envio do byte em dois *nibbles* (4 bits), economizando pinos de I/O do 8051.



(a) Módulo Arduino



(b) Conexões TX/RX



(c) Tráfego de Dados

Figura 1: Esquemático e conexões físicas entre Arduino, 8051 e Display LCD.

3 Implementação e Análise dos Códigos

A solução de software foi dividida em duas frentes: o programa em C++ embarcado no Arduino (responsável pelo *timing* e envio de dados) e o firmware em linguagem Assembly no 8051 (responsável por gerenciar as interrupções de hardware e o protocolo do display LCD).

3.1 Código do Arduino Uno (Transmissor)

O Arduino atua como mestre na comunicação serial. O desafio nesta etapa foi enviar o caractere correspondente à sílaba japonesa "Ko"(), frequentemente confundida com um "J"quadrado ou "C"invertido, presente na tabela estendida do controlador do LCD.

Para contornar o problema de codificação UTF-8 dos editores modernos, foi utilizado o comando `Serial.write()`, que transmite o byte cru (0xBA) correspondente ao endereço físico do caractere na ROM do display. Em seguida, a letra "A" é enviada no formato ASCII padrão.

```
1 void setup() {
2   Serial.begin(9600);
3 }
4
5 void loop() {
6   // Envia o byte exato (0xBA) que corresponde ao caractere Katakana 'Ko' no display
7   Serial.write(0xBA);
8
9   // Envia a letra "A" normal logo em seguida
10  Serial.print("A");
11
12  // Nota: Juntos, o 89S52 recebera a sequencia exata para formar " A "
13
14  delay(2000); // Aguarda 2 segundos antes de reenviar
15 }
```

Listing 1: Código fonte implementado no Arduino Uno

3.2 Código do 8051 (Receptor e Controlador do LCD)

No 8051, a UART foi configurada operando no Modo 1 (8 bits) em conjunto com o Timer 1 (modo de auto-reload de 8 bits) gerando a taxa de transmissão. O *baud rate* foi ajustado, e a interrupção serial (ISR_SERIAL) foi habilitada. O microcontrolador permanece ocioso (*Idle*) até que um byte seja recebido.

Quando a interrupção ocorre, o byte é resgatado do registrador SBUF e enviado diretamente para a rotina `LCD_SendByte`. Essa rotina divide a informação recebida no Acumulador (A) em dois *nibbles* de 4 bits para alimentar o LCD sincronizadamente com os pulsos do pino Enable (EN).

```
1 ;-----
2 ; DEFINICAO DE PINOS
3 ;-----
4 RS EQU P1.3      ; Register Select: 0=Comando, 1=Dado
5 RW EQU P1.2      ; Read/Write:      0=Escrita, 1=Leitura
6 EN EQU P1.1      ; Enable: borda de descida aciona o LCD
7
8 ;-----
9 ; VETOR DE RESET E INTERRUPTCOES
10 ;-----
11 ORG 0000H
12 JMP INICIO
13
14 ORG 0023H
15 JMP ISR_SERIAL
16
17 ;=====
18 ; INICIO - Configuracao geral do sistema
19 ;=====
20 ORG 0030H
21
22 INICIO:
23 ;--- Inicializa pinos de controle do LCD ---
24 CLR RS
25 CLR RW
```

```

26
27 ;--- Sequencia de inicializacao do LCD (modo 4 bits) ---
28 CALL LCD_FuncSet      ; Function Set: 4 bits, 2 linhas, fonte 5x8
29 CALL LCD_DispCtrl     ; Display Control: display on, cursor on, blink on
30 CALL LCD_EntryMode    ; Entry Mode: incrementa cursor, sem shift
31 CALL LCD_Limpa        ; Limpa display e retorna cursor
32
33 ;--- Configuracao da UART ---
34 MOV PCON, #80H        ; SMOD=1: dobra baud rate
35 MOV TMOD, #20H        ; Timer1 no modo 2 (auto-reload de 8 bits)
36 MOV TH1, #0E6H        ; Reload para 1200 bps @ 11.0592 MHz com SMOD=1
37 MOV SCON, #50H        ; Modo 1 (8 bits), REN habilitado
38 SETB TR1              ; Inicia Timer1
39
40 ;--- Habilita interrupcao serial e interrupcao global ---
41 SETB ES
42 SETB EA
43
44 ;=====
45 ; LACO PRINCIPAL
46 ;=====
47 MAIN_LOOP:
48     SJMP MAIN_LOOP
49
50 ;=====
51 ; ISR_SERIAL - Interrupcao de recepcao serial
52 ;=====
53 ISR_SERIAL:
54     JNB RI, ISR_SAIDA    ; Ignora se nao foi recepcao (ex: TX)
55     MOV A, SBUF          ; Le o byte recebido
56     CLR RI              ; Limpa flag de recepcao
57
58     SETB RS              ; RS=1: envia como dado (caractere)
59     CLR RW
60     CALL LCD_SendByte    ; Exibe no LCD
61
62 ISR_SAIDA:
63     RETI
64
65 ;=====
66 ; ROTINAS DE CONTROLE DO LCD
67 ;=====
68 LCD_Limpa:
69     CLR RS              ; RS=0: modo comando
70     CLR RW
71     MOV A, #01H
72     CALL LCD_SendByte
73     CALL Delay
74     RET
75
76 LCD_FuncSet:
77     ; Nibble alto: 0010 (seleciona modo 4 bits)
78     CLR P1.7
79     CLR P1.6
80     SETB P1.5
81     CLR P1.4
82     CALL LCD_Pulso
83     CALL Delay
84
85     ; Nibble baixo: 1000 (2 linhas, fonte 5x8)
86     SETB P1.7
87     CLR P1.6
88     CLR P1.5
89     CLR P1.4
90     CALL LCD_Pulso
91     CALL Delay
92     RET
93
94 LCD_DispCtrl:
95     ; Nibble alto: 0000
96     CLR P1.7
97     CLR P1.6
98     CLR P1.5

```

```

99     CLR    P1.4
100    CALL   LCD_Pulso
101
102    ; Nibble baixo: 1111
103    SETB   P1.7
104    SETB   P1.6
105    SETB   P1.5
106    SETB   P1.4
107    CALL   LCD_Pulso
108    CALL   Delay
109    RET
110
111 LCD_EntryMode:
112    ; Nibble alto: 0000
113    CLR    P1.7
114    CLR    P1.6
115    CLR    P1.5
116    CLR    P1.4
117    CALL   LCD_Pulso
118
119    ; Nibble baixo: 0110
120    CLR    P1.7
121    SETB   P1.6
122    SETB   P1.5
123    CLR    P1.4
124    CALL   LCD_Pulso
125    CALL   Delay
126    RET
127
128 LCD_Pulso:
129    SETB   EN
130    CLR    EN
131    RET
132
133 LCD_SendByte:
134    ; Nibble alto (bits 7..4)
135    MOV    C, ACC.7
136    MOV    P1.7, C
137    MOV    C, ACC.6
138    MOV    P1.6, C
139    MOV    C, ACC.5
140    MOV    P1.5, C
141    MOV    C, ACC.4
142    MOV    P1.4, C
143    CALL   LCD_Pulso
144
145    ; Nibble baixo (bits 3..0)
146    MOV    C, ACC.3
147    MOV    P1.7, C
148    MOV    C, ACC.2
149    MOV    P1.6, C
150    MOV    C, ACC.1
151    MOV    P1.5, C
152    MOV    C, ACC.0
153    MOV    P1.4, C
154    CALL   LCD_Pulso
155
156    CALL   Delay
157    RET
158
159 Delay:
160    MOV    R6, #20
161 Delay_L1:
162    MOV    R7, #250
163    DJNZ   R7, $
164    DJNZ   R6, Delay_L1
165    RET
166
167    END

```

Listing 2: Firmware em Assembly para o microcontrolador 8051

4 Resultados e Conclusão

A execução do código resultou na correta recepção via interrupção pela placa 8051. Os dados trafegaram pela UART e foram processados pelo display em tempo real. A utilização do endereço hexadecimal exato via `Serial.write(0xBA)` demonstrou-se eficaz para suprimir os problemas clássicos de *encoding* textuais, resultando na visualização exata pretendida.

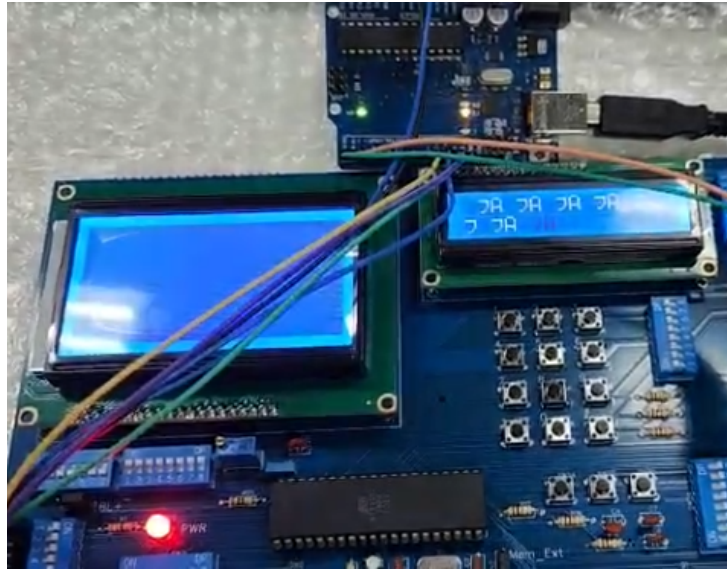


Figura 2: Exibição da string correspondente no Display LCD acionado em modo 4 bits.

Pode-se concluir que a divisão das responsabilidades (Arduino para geração/sequenciamento dos dados e 8051 para a gerência de baixo nível do hardware) é uma excelente estratégia arquitetural, permitindo fixar os conceitos de interfaceamento, registradores *SFRs* e rotinas de interrupção (ISR).

5 Referências

- [1] Slides de aula fornecidos pelo professor.